

Freight Rate Prediction

Machine Learning Engineer Assessment — Spotter | Ghadi Dababneh | Code: github.com/Ghadiiz/freight-rate-prediction

1. Summary

The goal is to predict the posted rate (USD) of freight loads. A gradient boosting model was trained on 48,000 labeled loads from January to October 2025 and evaluated with a **time-based split**, because the 12,000 loads to predict come from November and December 2025. On the most recent held-out months (September–October), the model has a mean absolute error of **\$107.22 (4.74%)**, 26% lower than a lane-based baseline.

2. Data and key findings

File	Rows	Period	Purpose
train_test.csv	48,000	Jan 1 – Oct 31, 2025	Labeled development data
validation.csv	12,000	Nov 1 – Dec 31, 2025	Final predictions (no rate)
december_chart_inputs.csv	31	Dec 1 – Dec 31, 2025	One fixed load, only the date changes

- **Validation is entirely in the future** relative to the training data, which determines the split strategy (Section 4).
- **The December file contains only six inputs:** pickup, delivery, distance, equipment, weight and date. It lacks *market_index*, *quote_signal* and the coordinates, so the model is restricted to features available in every file.
- **Distance is the strongest driver** of price; longer trips cost less per mile (correlation with rate per mile: -0.59).
- **Rates follow time patterns:** a seasonal rise toward a late-June peak, a flatter level from August to October, and a small weekly cycle (Thursday highest, weekends lowest).
- **New cities in validation:** eight cities never appear in training, affecting 1,447 validation loads (12%).

3. Data quality issues and fixes

Issue	Evidence	Fix
Negative weights (292 train, 145 validation)	Absolute values match the normal weight distribution (5,000–47,500 lb)	Sign error: absolute value applied to every file
Missing values (weight 300, market_index 374)	Under 1% of rows	Weight left missing (handled natively by the model); market_index not used
Extreme prices (\$57 to \$25,533)	e.g. \$14.13/mile on a 315-mile load whose quote was \$1.24/mile	653 loads (1.36%) priced below 0.5× or above 3× their own quote removed from training only
Date stored as text	Column type is text, not date	Converted to a date type

The outlier rule compares each price with the load's own quote ($\text{posted_rate} / (\text{distance} \times \text{quote_signal})$) rather than using a fixed per-mile limit, so legitimately expensive short trips are not removed. *quote_signal* is used only to detect errors, not as a model input. Validation rows are never removed.

4. Validation and data split approach

A random split would mix later loads into training while testing on earlier ones, letting the model effectively see the future and producing an over-optimistic score. Since the real task is to predict November–December from January–October, the evaluation imitates it:

Set	Period	Rows	Treatment
Train	Jan 1 – Aug 31, 2025	37,977	Price outliers removed
Test	Sep 1 – Oct 31, 2025	9,523 (20%)	Left untouched, outliers included

- **Why the most recent months:** September–October are closest in time to the loads being predicted, so they give the most relevant estimate.
- **Why the test set is not cleaned:** the real validation data contains messy loads too; cleaning the test set would overstate accuracy.
- **Final model:** after evaluation, the model is retrained on all labeled data (Jan–Oct) so it learns from the months closest to November–December.

5. Model

Features: pickup city, delivery city and equipment (categorical); distance and weight; and three date features — day of week, day of month and month.

Model: scikit-learn's *HistGradientBoostingRegressor* with absolute-error loss (learning rate 0.05, 500 iterations). Gradient-boosted trees are a strong default for tabular data; this implementation handles categorical features and missing values natively, and the absolute-error loss limits the influence of any remaining extreme prices. Cities unseen in training are treated as missing, so the model falls back on distance, equipment and date.

Feature choice tested: adding *market_index* and *quote_signal* did not improve the test error (it rose slightly), so they were dropped. This keeps one model valid for both the validation file and the December file.

6. Results

Model (tested on Sep–Oct 2025)	MAE	MAPE
Baseline: median rate per mile of the same lane and equipment	\$144.85	6.31%
Gradient boosting (final approach)	\$107.22	4.74%

MAE is the average dollar error per load; MAPE is the average percentage error. Both output files passed validation with the provided *score.py*.

7. December 2025 prediction chart

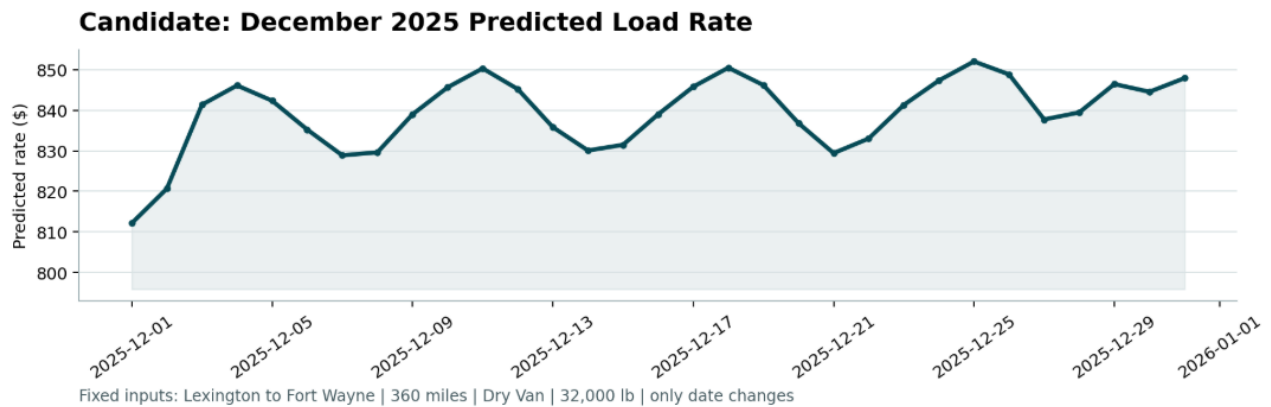


Chart produced by the provided score.py. Fixed load: Lexington → Fort Wayne, 360 miles, Dry Van, 32,000 lb.

Predictions range from about \$812 to \$852 (roughly \$2.26–2.37 per mile). The weekly cycle matches the pattern found in the data: peaks fall on Thursdays (Dec 4, 11, 18, 25) and dips on Sundays (Dec 7, 14, 21, 28). December 1 is the lowest point, reflecting the lower rates the model learned for the first days of a month.

8. Limitations and next steps

- **No December in training:** the model cannot learn holiday effects. December 25 appears as a peak simply because it is a Thursday. More historical data covering past holiday seasons, or explicit holiday features, would address this.
- **No extrapolation of trends:** tree models predict at the level of the most recent data and do not extend trends into the future.
- **Unseen cities:** 12% of validation loads involve a city absent from training. Coordinates could help generalization if they were available in every input file.